# ITA-0607 MACHINE LEARNING

**P.C. BHARATH SIMHA REDDY**

**192424361**

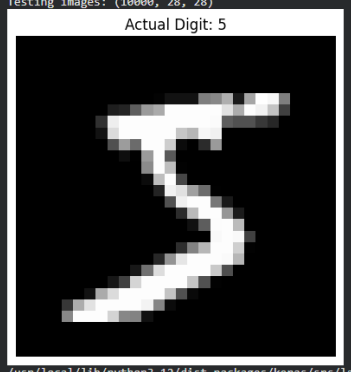
## GRAY:

```python
# Step 1: Import libraries
import tensorflow as tf
import matplotlib.pyplot as plt
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense
# Step 2: Load MNIST handwritten digit dataset
(X_train, y_train), (X_test, y_test) = tf.keras.datasets.mnist.load_data()
print("Training images:", X_train.shape)
print("Testing images:", X_test.shape)
# Step 3: Display one image
plt.imshow(X_train[0], cmap="gray")
plt.title("Actual Digit: " + str(y_train[0]))
plt.axis("off")
plt.show()
# Step 4: Normalize pixel values
# Original pixel values are between 0 and 255
# Convert them into values between 0 and 1
X_train = X_train / 255.0
X_test = X_test / 255.0
# Step 5: Add channel dimension
# CNN expects: height, width, channel
X_train = X_train.reshape(-1, 28, 28, 1)
X_test = X_test.reshape(-1, 28, 28, 1)
# Step 6: Build the CNN model
model = Sequential([
    # Detect simple features such as edges and curves
    Conv2D(
        filters=32,
        kernel_size=(3, 3),
        activation="relu",
        input_shape=(28, 28, 1)
    ),
    # Reduce the image size
    MaxPooling2D(pool_size=(2, 2)),
    # Convert the feature maps into one-dimensional form
    Flatten(),
    # Learn important combinations of features
```


```python
# Step 1: Import libraries
import tensorflow as tf
import matplotlib.pyplot as plt
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense
# Step 2: Load MNIST handwritten digit dataset
(X_train, y_train), (X_test, y_test) = tf.keras.datasets.mnist.load_data()
print("Training images:", X_train.shape)
print("Testing images:", X_test.shape)
# Step 3: Display one image
plt.imshow(X_train[0], cmap="gray")
plt.title("Actual Digit: " + str(y_train[0]))
plt.axis("off")
plt.show()
# Step 4: Normalize pixel values
# Original pixel values are between 0 and 255
# Convert them into values between 0 and 1
X_train = X_train / 255.0
X_test = X_test / 255.0
# Step 5: Add channel dimension
# CNN expects: height, width, channel
X_train = X_train.reshape(-1, 28, 28, 1)
X_test = X_test.reshape(-1, 28, 28, 1)
# Step 6: Build the CNN model
model = Sequential([
    # Detect simple features such as edges and curves
    Conv2D(
        filters=32,
        kernel_size=(3, 3),
        activation="relu",
        input_shape=(28, 28, 1)
    ),
    # Reduce the image size
    MaxPooling2D(pool_size=(2, 2)),
    # Convert the feature maps into one-dimensional form
    Flatten(),
    # Learn important combinations of features
```

# OUTPUT:

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 2s 0us/step
Training images: (60000, 28, 28)
Testing images: (10000, 28, 28)
```




```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequer
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential"
```

| Layer (type) | Output Shape | Param # |
|---|---|---|
| conv2d (Conv2D) | (None, 26, 26, 32) | 320 |
| max_pooling2d (MaxPooling2D) | (None, 13, 13, 32) | 0 |
| flatten (Flatten) | (None, 5408) | 0 |
| dense (Dense) | (None, 64) | 346,176 |
| dense_1 (Dense) | (None, 10) | 650 |

```
Total params: 347,146 (1.32 MB)
Trainable params: 347,146 (1.32 MB)
Non-trainable params: 0 (0.00 B)
Epoch 1/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 33s 19ms/step - accuracy: 0.9450 - loss: 0.1823 - val_accuracy: 0.9828 - val_loss: 0.0627
Epoch 2/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 40s 18ms/step - accuracy: 0.9806 - loss: 0.0632 - val_accuracy: 0.9812 - val_loss: 0.0621
Epoch 3/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 41s 18ms/step - accuracy: 0.9866 - loss: 0.0432 - val_accuracy: 0.9860 - val_loss: 0.0516
Epoch 4/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 30s 18ms/step - accuracy: 0.9902 - loss: 0.0310 - val_accuracy: 0.9778 - val_loss: 0.0780
Epoch 5/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 29s 17ms/step - accuracy: 0.9927 - loss: 0.0224 - val_accuracy: 0.9878 - val_loss: 0.0589
313/313 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - accuracy: 0.9866 - loss: 0.0415
Test Accuracy: 0.9865999817848206
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 78ms/step
```

```
Test Accuracy: 0.9865999817848206
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 78ms/step
```